

วันที่ 1

พื้นฐาน MiniZinc และ Optimization

สำหรับพนักงานอุตสาหกรรม — ไม่ต้องมีพื้นฐานการเขียนโปรแกรม

🕒 ระยะเวลา

6 ชั่วโมง

🌞 ช่วงเช้า

09:00–12:00

🌙 ช่วงบ่าย

13:00–16:00

กำหนดการวันที่ 1

09:00 – 09:45

 **Optimization คืออะไร?***แนวคิด, ตัวอย่างในอุตสาหกรรม, ทำไมต้อง MiniZinc*

09:45 – 10:15

 **ติดตั้งและเริ่มต้น IDE***ดาวน์โหลด, ทำความรู้จัก IDE, รัน Hello World*

10:15 – 11:00

 **โมเดลแรก: Map Coloring***Variables, Constraints, solve satisfy*

11:00 – 12:00

 **Workshop 1A***ฝึกแก้โจทย์ Map Coloring ด้วยตนเอง*

13:00 – 14:00

 **Arithmetic Optimization***maximize/minimize, arithmetic constraints*

14:00 – 15:00

 **Data Files & Parameters***แยก .mzn และ .dzn, assert()*

15:00 – 16:00

 **Workshop 1B***Production Planning — โรงงานผลิตสินค้า*

Optimization คืออะไร?

"การหาคำตอบที่ดีที่สุด ภายใต้เงื่อนไขที่กำหนด"



จัดตารางกะ

หาตารางกะพนักงานที่ครบทุก shift และ
ยุติธรรมที่สุด



จัดเส้นทางรถ

หาเส้นทางส่งสินค้าที่สั้นที่สุดหรือถูกที่สุด



จัดสินค้าลงกล่อง

ใช้กล่องน้อยที่สุดในการบรรจุสินค้า
ทั้งหมด

วิธีดั้งเดิม vs. Optimization

😞 วิธีดั้งเดิม (ลองทุกคำตอบ)

- ▶ พนักงาน 10 คน, 3 กะ → ลองทุกการจัดเรียง
- ▶ $= 3^{10} = 59,049$ คำตอบ!
- ▶ ใช้เวลานาน, อาจพลาดคำตอบที่ดีที่สุด
- ▶ เมื่อขนาดปัญหาใหญ่ขึ้น → เป็นไปไม่ได้

✅ Constraint Programming

- ✓ บอก solver ว่า 'อยากได้อะไร'
- ✓ ไม่ต้องบอก 'วิธีหา'
- ✓ Solver ใช้ algorithm ฉลาด ตัดสาขาที่เป็นไปไม่ได้
- ✓ หาคำตอบที่ดีที่สุด (proved optimal)

MiniZinc คืออะไร?

ภาษาสำหรับ อธิบายปัญหา — ไม่ใช่ภาษาโปรแกรมทั่วไป



Modelling Language

บอกว่า 'ปัญหาคืออะไร' ไม่ใช่ 'แก้อย่างไร'



ใช้ในวิจัยและอุตสาหกรรม

มหาวิทยาลัย, โรงงาน, โลจิสติกส์, การแพทย์



ต่อกับ Solver ได้หลายตัว

Gecode, OR-Tools, Chuffed, Gurobi, CPLEX



ฟรีและ Open Source

minizinc.org — ดาวน์โฮลด์ได้เลย

ติดตั้ง MiniZinc IDE

- 1** **ดาวน์โหลด** ไปที่ minizinc.org → Download → เลือก Windows / macOS / Linux
- 2** **ติดตั้ง** รันไฟล์ installer → กด Next จนเสร็จ
- 3** **เปิด MiniZinc IDE** เปิดโปรแกรม → จะเห็น editor, output panel, solver menu
- 4** **ทดสอบ** สร้างไฟล์ใหม่ → พิมพ์ solve satisfy; → กด Run ▶



โครงสร้างพื้นฐานของโมเดล MiniZinc

Parameters

ค่าคงที่ที่เรารู้
(กำหนดใน model หรือ .dzn)

```
int: n = 5;
```

Decision Variables

ค่าที่ Solver
ต้องหาให้เรา

```
var 0..n: x;
```

Constraints

กฎที่ต้องเป็นจริง

```
constraint x > 0;
```

Solve

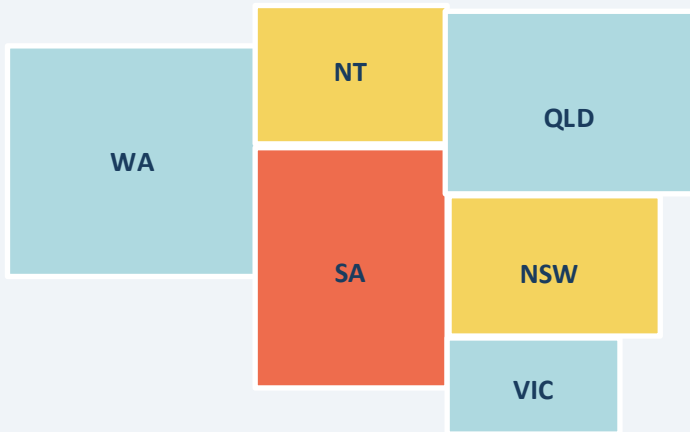
เป้าหมาย
(satisfy/min/max)

```
solve satisfy;
```

ตัวอย่างแรก: ระบายสีแผนที่ออสเตรเลีย

โจทย์: ระบายสีแผนที่ 7 รัฐของออสเตรเลีย โดยรัฐที่มีพรมแดนติดกันต้องมีสีต่างกัน

แผนที่ออสเตรเลีย



รัฐที่ติดกัน (ต้องสีต่างกัน):

- ▶ $WA \leftrightarrow NT, SA$
- ▶ $NT \leftrightarrow SA, QLD$
- ▶ $SA \leftrightarrow QLD, NSW, VIC$
- ▶ $QLD \leftrightarrow NSW$
- ▶ $NSW \leftrightarrow VIC$
- ▶ TAS — (ไม่ติดกับใคร)

โค้ด MiniZinc: aust.mzn

```
% Parameter
int: nc = 3;

% Decision Variables
var 1..nc: wa;   var 1..nc: nt;
var 1..nc: sa;   var 1..nc: q;
var 1..nc: nsw;  var 1..nc: v;
var 1..nc: t;

% Constraints
constraint wa != nt;
constraint wa != sa;
constraint nt != sa;  nt != q;
constraint sa != q;   sa != nsw;  sa != v;

% Solve
solve satisfy;
```

◀ กำหนดจำนวนสีที่ใช้ได้

◀ แต่ละรัฐคือ 1 ตัวแปร
ค่าคือหมายเลขสี (1-3)

◀ รัฐที่ติดกัน
ต้องสีต่างกัน

◀ หาคำตอบที่ตรงเงื่อนไข

Parameters vs. Decision Variables

Parameters (ค่าคงที่)

- ▶ เราเป็นคนกำหนดค่า
- ▶ ไม่เปลี่ยนแปลงระหว่าง solve
- ▶ Solver ไม่แตะต้อง
- ▶ ตัวอย่าง:

```
int: nc = 3;  
int: capacity = 100;
```

Decision Variables (ตัวแปร)

- ▶ Solver เป็นคนหาค่า
- ▶ ต้องกำหนดชนิด (int/bool/float)
- ▶ ต้องกำหนด domain (ขอบเขต)
- ▶ ตัวอย่าง:

```
var 1..nc: wa;  
var 0..100: amount;
```

Constraints — กฎที่ต้องเป็นจริง

`constraint <Boolean expression>;` → ต้องเป็น true เสมอ!

Relational Operators:

`=` หรือ `==` → เท่ากับ

`!=` → ไม่เท่ากับ

`<` และ `>` → น้อยกว่า/มากกว่า

`<=` และ `>=` → น้อยกว่าเท่ากับ/มากกว่าเท่ากับ

Boolean Operators:

`&` AND — ต้องเป็นจริงทั้งคู่

`|` OR — เป็นจริงอย่างน้อยหนึ่งอย่าง

`not` NOT — ต้องเป็นเท็จ

`->` IMPLIES — ถ้า A จริง ต้อง B จริง

solve satisfy — และผลลัพธ์

`solve satisfy;`

หาคำตอบที่ตรงทุก constraint
(ไม่สนใจว่าดีแค่ไหน)

`solve maximize obj;`

หาคำตอบที่ทำให้ obj
มีค่ามากที่สุด

`solve minimize obj;`

หาคำตอบที่ทำให้ obj
มีค่าน้อยที่สุด

```
wa=3  nt=2  sa=1  ...  -----  =====
```

= คำตอบ

--- = พบคำตอบ

=== = optimal (min/max)



Workshop 1A — Map Coloring

เวลา: 60 นาที | ไฟล์: `workshop1a_starter.mzn`

1

รันโมเดลที่มีอยู่

เปิด `workshop1a_starter.mzn` → กด Run ► → ดูผลลัพธ์ แต่ละรัฐได้สีอะไร?

2

ลด `nc = 2`

เปลี่ยน `nc = 3` → `nc = 2` แล้วรันใหม่ → เกิดอะไรขึ้น? ทำไม? (UNSATISFIABLE คืออะไร?)

3

เพิ่มรัฐใหม่ 'act'

Uncomment บรรทัด `act` → เพิ่ม constraint `act != nsw`; และ `act != v`; → รันและตรวจสอบ

4

(ท้าทาย) Minimize

เปลี่ยน `solve satisfy` เป็น `solve minimize wa+nt+sa+q+nsw+v+t` → ผลเปลี่ยนไปไหม?

ช่วงบ่าย

Arithmetic Optimization และ Data Files

- solve maximize / minimize

- Arithmetic operators

- Data files (.dzn)

- assert()

ตัวอย่าง: Maximize กำไรการอบเค้ก

โจทย์: อบเค้กกล้วย (กำไร 400 บาท) และเค้กช็อก (กำไร 450 บาท) ให้ได้กำไรสูงสุด
ภายใต้ข้อจำกัดวัตถุดิบ: แป้ง, กล้วย, น้ำตาล, เนย, โกโก้

วัตถุดิบที่ต้องการ:

วัตถุดิบ	เค้กกล้วย (b)	เค้กช็อกโกแลต (c)	ที่มีทั้งหมด
แป้ง (g)	250	200	4,000
กล้วย (ลูก)	2	0	6
น้ำตาล (g)	75	150	2,000
เนย (g)	100	150	500
โกโก้ (g)	0	75	500
กำไร (บาท)	400	450	—

โค้ด MiniZinc: cakes.mzn

```
% Decision Variables
var 0..100: b; % เค้กกล้วย
var 0..100: c; % เค้กช็อก

% Constraints
constraint 250*b + 200*c <= 4000;
constraint 2*b <= 6;
constraint 75*b + 150*c <= 2000;
constraint 100*b + 150*c <= 500;
constraint 75*c <= 500;

% Objective
solve maximize 400*b + 450*c;

output ["b=", show(b), ", c=", show(c)];
```

◀ ตัวแปรเป็น integer
กำหนด bound ให้ solver

◀ แต่ละ constraint
คือ resource ที่จำกัด

◀ objective function
ที่ต้อง maximize

◀ \ (expr) = string interpolation

Arithmetic Operators ใน MiniZinc

+

บวก

 $a + b$

-

ลบ

 $a - b$

*

คูณ

 $3 * x$

div

หารจำนวนเต็ม

 $10 \text{ div } 3 \rightarrow 3$

mod

เศษการหาร

 $10 \text{ mod } 3 \rightarrow 1$

abs(x)

ค่าสัมบูรณ์

 $\text{abs}(-5) \rightarrow 5$ 

ใน MiniZinc ตัวเลข a/b จะเป็น floating point เสมอ ถ้าต้องการหารจำนวนเต็มให้ใช้ $a \text{ div } b$

แยก Model (.mzn) และ Data (.dzn)

ทำไมต้องแยก? → โมเดลเดิม + ข้อมูลต่างกัน = ผลลัพธ์ต่างกัน — ไม่ต้องแก้โค้ดทุกครั้ง!

cakes_param.mzn (Model)

```
% รับค่าจาก data file

int: flour;

int: banana;

var 0..100: b;

constraint 250*b <= flour;
```

pantry.dzn (Data)

```
% ค่าจริงที่ใช้วันนี้

flour  = 4000;

banana = 6;

sugar  = 2000;

butter = 500;

cocoa  = 500;
```

```
$ minizinc 03_cakes_param.mzn 03_pantry.dzn
```

assert() — ตรวจสอบข้อมูลก่อน Solve

`assert(condition, error_message)` → ถ้า condition เป็น false จะ ERROR พร้อมแสดง message

```
% ตรวจสอบว่าข้อมูลสมเหตุสมผลก่อน solve

constraint assert(flour >= 0, "แป้งต้องไม่ติดลบ");

constraint assert(banana >= 0, "กล้วยต้องไม่ติดลบ");

constraint assert(capacity > 0, "ความจุต้องมากกว่า 0");
```

ทำไมต้องใช้ assert?

- ✓ ป้องกัน garbage-in garbage-out
- ✓ ช่วย debug ข้อมูลผิดพลาดได้เร็ว
- ✓ เป็น best practice สำหรับโมเดลที่ใช้กับ data file
- ✗ Error message ที่ชัดเจนดีกว่า solver ที่ crash

output — แสดงผลลัพธ์

```
output [
    "แค่กล้วย: ", show(b), "\n",
    "กำไรรวม:   ", show(400*b + 450*c), "\n",
    "แค่กล้วย: \b) ขึ้น\n" % string interpolation
];
```

ฟังก์ชัน Output ที่มีประโยชน์:

<code>show(x)</code>	แสดงค่าของ x เป็น string
<code>show_int(n, x)</code>	แสดง integer ในความกว้าง n ตัวอักษร
<code>show_float(n, d, x)</code>	แสดง float ความกว้าง n ทศนิยม d ตำแหน่ง
<code>\(expr)</code>	String interpolation — ใส่ใน double quotes
<code>\n \t</code>	Newline และ Tab



Workshop 1B — Production Planning

เวลา: 60 นาที | ไฟล์: `workshop1b_starter.mzn`

โจทย์:

ผลิตภัณฑ์ A: แรงงาน 3 ชม. + วัสดุดิบ 5 กก., กำไร 800 บาท/หน่วย

ผลิตภัณฑ์ B: แรงงาน 5 ชม. + วัสดุดิบ 2 กก., กำไร 1,200 บาท/หน่วย

ข้อจำกัด: แรงงาน 120 ชม./วัน, วัสดุดิบ 150 กก./วัน

- | | | |
|---|------------------------------------|---|
| 1 | กำหนด Decision Variables : | <code>var ???: a;</code> และ <code>var ???: b;</code> — bound ควรเป็นเท่าไร? |
| 2 | เพิ่ม Constraints : | constraint สำหรับแรงงาน ($3*a + 5*b \leq 120$) และวัสดุดิบ ($5*a + 2*b \leq 150$) |
| 3 | กำหนด Objective : | <code>solve maximize 800*a + 1200*b;</code> แล้วรันและตรวจสอบคำตอบ |
| 4 | เพิ่ม minimum requirement : | constraint <code>a >= 5;</code> และ <code>b >= 3;</code> — คำตอบเปลี่ยนไปไหม? |
| 5 | สร้าง Data File : | ย้ายค่าทั้งหมดไป <code>workshop1b_data.dzn</code> แล้วรัน: <code>minizinc ... model.mzn data.dzn</code> |

สรุปวันที่ 1 — สิ่งที่ต้องจำ



Parameters vs. Decision Variables

Parameters เราระบุ | Variables Solver หาให้ | ใช้ var ก่อนประกาศ variable



Constraint คือกฎ

ต้องเป็นจริงทุกข้อพร้อมกัน | ถ้าขัดกัน → UNSATISFIABLE



solve satisfy / maximize / minimize

บอก solver ว่าต้องการอะไร | maximize/minimize ต้องมี objective expression



Data Files (.dzn)

แยก model กับ data ทำให้ reuse ได้ | ใช้ assert() ตรวจสอบข้อมูล

แหล่งเรียนรู้เพิ่มเติม

 **docs.minizinc.dev**

Tutorial อย่างเป็นทางการ — อ้างอิงในหลักสูตรนี้

 **MiniZinc Playground**

ทดลองเขียน MiniZinc ออนไลน์ไม่ต้องติดตั้ง

 **Global Constraints Catalog**

รายการ global constraints ทั้งหมดพร้อมตัวอย่าง

 **MiniZinc Forum**

ถามคำถาม, แชร์โมเดล, เรียนรู้จากชุมชน

ไฟล์ประกอบวันที่ 1: day1/ folder — มีตัวอย่างทั้งหมดและ Workshop starter + solution